



INSTITUT SUPÉRIEUR POLYTECHNIQUE DE MADAGASCAR

Fahaizana · Fampandrosoana · Fihavanana

Corrigé — Exercices de révision Java

Interfaces, généricité, collections et API Stream — L2 Informatique, 2025–2026

RABOANARY Heriniaina Andry, *Enseignant chercheur*

Réponses attendues et codes de référence, compilés et exécutés sous Java 21 ; les blocs sur fond vert clair sont les sorties console obtenues. Sources complètes : dossier **Révisions-2026** du dépôt <https://github.com/AndryRAB/JAVA-L2>.

Exercice 1 — Compréhension

1) Définitions

Le mot central, en italique, est ce qui rend chaque définition juste ou fausse.

- (a) **Classe générique**. Classe paramétrée par un ou plusieurs *types formels*, notés entre chevrons, que l'utilisateur fixe à la déclaration : on écrit le code une fois, il vaut pour tous les types.
- (b) **Méthode abstraite**. Méthode déclarée avec sa *signature complète mais sans corps* ; elle n'existe que dans une classe *abstract* ou une interface et oblige toute sous-classe concrète à l'implémenter.
- (c) **Interface**. Type qui déclare un *contrat* : un ensemble de signatures que la classe s'engage à fournir. Pas d'état d'instance ; une classe peut en implémenter plusieurs.
- (d) **Interface fonctionnelle**. Interface possédant *exactement une* méthode abstraite (SAM) ; cette unicité permet la lambda et la référence de méthode.
- (e) **Classe anonyme**. Classe *sans nom*, déclarée et instanciée en une seule expression. Elle peut porter des champs — donc un état — et plusieurs méthodes.
- (f) **Expression lambda**. *Notation abrégée* implémentant une interface fonctionnelle : paramètres, flèche, corps. Aucun état propre.
- (g) **Programmation fonctionnelle**. Style où le calcul s'exprime par *application et composition de fonctions*, plutôt que par une suite d'instructions qui modifient un état.
- (h) **Fonction d'ordre supérieur**. Fonction qui prend une fonction *en paramètre* ou qui en renvoie une — conséquence du fait que les fonctions sont devenues des valeurs.

2.a) Généricité : quand l'erreur est-elle détectée ?

Les deux versions contiennent la même faute — on range une `String` et on relit un `Integer` — mais pas au même moment.

Version A (sans générique) : le code compile, car `sortir()` rend un `Object` et le transtypage est une promesse que le programmeur est libre de faire. La faute n'apparaît qu'à l'exécution. *Version B (avec générique)* : le code ne compile pas.

```
A : Exception in thread "main" java.lang.ClassCastException:
    class java.lang.String cannot be cast to class java.lang.Integer
B : error: incompatible types: String cannot be converted to Integer
    Integer n = b.sortir();
```

Conclusion. La généricité ne rend pas le code plus sûr par magie : elle **déplace la détection de l'erreur**, de l'exécution vers la compilation, et supprime au passage le transtypage explicite.

2.b) Expression lambda vs classe anonyme

	Classe anonyme	Lambda
Écriture	5 lignes, nom de méthode répété	une ligne, l'intention seule
Types des paramètres	à écrire	déduits du type cible
this	l'instance anonyme	l'objet englobant
État propre (champs)	possible	impossible
Nombre de méthodes	plusieurs	une seule
Cible	toute interface ou classe	interfaces fonctionnelles

Ce que la lambda ne peut plus faire : implémenter une interface qui déclare deux méthodes abstraites — il n'y a plus de méthode unique à laquelle rattacher son corps.

```
interface Deplacable { void avancer(int pas); void reculer(int pas); }
Deplacable impossible = pas -> System.out.println(pas); // refuse
```

Le compilateur répond `Deplacable is not a functional interface: multiple non-overriding abstract methods found`. Seule la classe anonyme convient alors, puisqu'elle nomme chaque méthode. Une lambda ne peut pas non plus étendre une classe abstraite, ni se désigner elle-même par `this`.

2.c) Comparator vs Comparable

`Comparable` inscrit l'ordre *dans* la classe : ordre naturel, unique, et le définir suppose de pouvoir modifier son code. `Comparator` définit l'ordre *à l'extérieur* : un objet séparé qu'on passe à `sort`, à `max` ou au constructeur d'une `PriorityQueue`. Trier une classe dont on ne possède pas le code source est donc impossible avec `Comparable` et immédiat avec `Comparator` — le cas est courant : `String` est `final`, `LocalDate` appartient au JDK.

```
Collections.sort(mots); // ordre naturel de String
mots.sort(Comparator.comparingInt(String::length)); // ordre externe
```

Sorties : [banane, fraise, kiwi, pomme] puis [kiwi, pomme, banane, fraise]. Second argument, valable même quand la classe nous appartient : un type n'a qu'un seul ordre naturel, mais on peut lui associer autant de `Comparator` que de critères.

2.d) Boucle vs pipeline

Les deux fragments produisent la même `Map` (vérifié à l'exécution : `true`). *La boucle rend visible le déroulement* — l'ordre des opérations, l'accumulateur qui se remplit, le test qui décide : c'est la version qu'on parcourt au débogueur. *Le pipeline rend visible l'intention* : `filter` puis `groupingBy` annoncent *ce que l'on veut* sans dire comment y parvenir.

Ce qui disparaît : la variable mutable et l'itération explicite ; mais aussi la sortie en cours de route par `break` ou `return`, l'indice de position, et le remplissage de plusieurs accumulateurs dans le même parcours.

Où *la boucle reste préférable* : interruption sur condition complexe, alimentation de plusieurs structures à la fois, corps susceptible de lever une exception vérifiée, ou effet de bord assumé (écriture dans un fichier, journalisation).

Exercice 2 — Application de la classe File de Priorité

2.1) La classe Smartphone

```
public class Smartphone implements Comparable<Smartphone> {

    private final String marque;
    private final int capaciteBatterie; // en mAh
    private final double tailleEcran; // en pouces
    private final double prix; // en euros
    private final int anneeSortie;

    // (a) constructeur affectant les cinq attributs, puis un accesseur par attribut :
    // getMarque(), getBatterie(), getEcran(), getPrix(), getAnnee()

    // (b) deux methodes au choix
    public int age(int anneeCourante) { return anneeCourante - anneeSortie; }
    public double densiteBatterie() { return capaciteBatterie / tailleEcran; }

    // (c) representation textuelle
    @Override public String toString() {
```

```

    return String.format(Locale.US, "%s (%d) - %d mAh, %.1f\", %.2f EUR",
                           marque, anneeSortie, capaciteBatterie, tailleEcran, prix);
}

// (d) ordre naturel : priorite au plus recent -> arguments inverses
@Override public int compareTo(Smartphone autre) {
    return Integer.compare(autre.anneeSortie, this.anneeSortie);
}
}

```

Le point décisif est le *sens* de la comparaison : le plus récent doit être prioritaire, donc `compareTo` renvoie une valeur négative lorsque l'année de `this` est la plus grande — les arguments de `Integer.compare` sont inversés par rapport à l'ordre croissant habituel.

Exécution d'une `PriorityQueue<Smartphone>` remplie du catalogue de l'exercice 4, vidée par `poll()` :

```

Apple (2025) - 3300 mAh, 6.1", 1099.00 EUR
Samsung (2025) - 4500 mAh, 6.1", 899.00 EUR
Xiaomi (2024) - 5000 mAh, 6.7", 399.00 EUR
Nokia (2023) - 5000 mAh, 6.5", 249.00 EUR
Nokia (2022) - 4000 mAh, 6.5", 179.00 EUR

```

Trois points à surveiller : `this.anneeSortie - autre.anneeSortie` (mauvais sens, et la soustraction d'entiers peut déborder); `(int)(this.prix - autre.prix)` pour un prix (la conversion tronque : deux prix distants de moins d'un euro deviennent égaux); l'oubli de implements `Comparable<Smartphone>`, qui fait lever une `ClassCastException` au premier ajout dans la file.

Enfin, une `PriorityQueue` est un tas, pas une liste triée : seule la tête est garantie prioritaire. Un parcours `for-each` donne l'ordre interne du tableau — sur ce catalogue, Apple 2025, Xiaomi 2024, Samsung 2025, Nokia 2022, Nokia 2023 — et non l'ordre de priorité. Pour obtenir tous les éléments dans l'ordre, il faut vider la file par `poll()` ou trier une copie.

2.2.a) Ordre alphabétique des marques

```

@Override public int compareTo(Smartphone autre) {
    return this.marque.compareTo(autre.marque); // String est déjà Comparable
}

```

`compareTo` sur les chaînes est sensible à la casse; `compareToIgnoreCase` corrige ce point.

2.2.b) Autonomie décroissante, puis écran décroissant, puis prix croissant

```

@Override public int compareTo(Smartphone autre) {
    int c = Integer.compare(autre.capaciteBatterie, this.capaciteBatterie); // 1er critere
    if (c != 0) return c;
    c = Double.compare(autre.tailleEcran, this.tailleEcran); // 2e critere
    if (c != 0) return c;
    return Double.compare(this.prix, autre.prix); // 3e critere
}

```

Un critère par bloc; on ne passe au suivant que si le précédent donne 0. Le sens s'obtient en inversant ou non les arguments, jamais en changeant le signe du résultat à la main. Ordre obtenu : Xiaomi 2024 (5000 mAh, 6,7 ") puis Nokia 2023 (5000 mAh, 6,5 "), Samsung 2025, Nokia 2022, Apple 2025. Le troisième critère n'est pas départagé par ce jeu de données : aucun couple n'a à la fois la même batterie et le même écran.

Les mêmes ordres s'obtiennent sans toucher à `Smartphone`, en passant un `Comparator` au constructeur de la file :

```

new PriorityQueue<>(Comparator.comparing(Smartphone::getMarque));

Comparator<Smartphone> parAutonomie =
    Comparator.comparingInt(Smartphone::getBatterie).reversed()
        .thenComparing(Comparator.comparingDouble(Smartphone::getEcran).reversed())
        .thenComparingDouble(Smartphone::getPrix);

```

Attention : `.reversed()` ne s'applique qu'au comparateur construit *jusqu'à ce point* — l'inversion du deuxième critère s'écrit donc à l'intérieur du `thenComparing`.

Exercice 3 — Indexation par une chaîne de caractères

3.1) TreeMap et HashMap

	HashMap	TreeMap
get/put/containsKey	$O(1)$ en moyenne	$O(\log n)$ garanti
Ordre d'itération	aucun ordre garanti	clés triées (naturel ou Comparator)
Contrainte sur les clés	hashCode()/equals() cohérents	clés Comparable ou comparateur
Clé null	acceptée	refusée : NullPointerException
Interface	Map	NavigableMap : firstKey, headMap...
Quand la choisir	accès par clé le plus rapide	parcours ordonné, requêtes par intervalle

Sur trois clés « a », « b », « c », un HashMap les affiche souvent dans l'ordre alphabétique : pure coïncidence de hachage. « HashMap conserve l'ordre d'insertion » est faux — c'est LinkedHashMap; contre-exemple sur clés réelles en 4.13.

3.2) Pourquoi une Map plutôt qu'une liste

Le HashMap calcule directement, à partir du code, la case où se trouve le titre : accès en temps constant en moyenne. Une ArrayList doit parcourir les éléments un à un — sur 100 000 livres, quelques opérations contre 50 000 en moyenne. Trier la liste par code accélérerait la *recherche* (dichotomie, $O(\log n)$), mais chaque *insertion* retomberait à $O(n)$: il faut décaler les éléments pour garder l'ordre. Une liste triée ne convient donc qu'à un catalogue constitué une fois puis seulement consulté; dès qu'il évolue, il faut une structure rapide des deux côtés : la Map de hachage, ou un TreeMap en $O(\log n)$ des deux côtés.

3.3 et 3.4) La classe

```
public class Bibliotheque {
    private final Map<String, String> titresParCode = new HashMap<>();

    public void ajouter(String code, String titre) { titresParCode.put(code, titre); }

    public String titreDe(String code) {
        return titresParCode.getOrDefault(code, "code inconnu"); // MAT-204 -> Analyse numerique
    } // ZZZ-999 -> code inconnu
    public int taille() { return titresParCode.size(); }
}
```

Deux points à valoriser : la déclaration par l'interface Map et non par HashMap, et le traitement explicite du code absent (valeur par défaut, ou Optional<String> laissé à l'appelant).

3.5) Conception : deux informations, deux recherches

Une Map associe *une* clé à *une* valeur; vouloir un troisième paramètre revient à lui demander de porter une information qu'elle n'a pas à porter. Deux décisions séparées résolvent le problème.

Décision 1 — la valeur devient un objet. On crée une classe Livre (code, titre, auteur) et l'on indexe Map<String, Livre> : ajouter demain l'éditeur ne changera plus la signature de la Map.

Décision 2 — la seconde recherche exige un second index. Retrouver les livres d'un auteur en parcourant tous les livres coûte $O(n)$, ce que l'énoncé demande d'éviter. On ajoute donc un *index inversé* Map<String, List<String>> associant chaque auteur à la liste des codes de ses livres — un auteur ayant plusieurs livres, la valeur est nécessairement une collection.

Les deux Map décrivent les mêmes données sous deux angles : elles doivent être modifiées *ensemble*. D'où leur *private*, et l'obligation de passer par les méthodes de la classe.

3.6) La classe complète

```
public class Bibliotheque {

    private final Map<String, Livre> livresParCode = new HashMap<>();
    private final Map<String, List<String>> codesParAuteur = new HashMap<>();

    public void ajouter(Livre livre) {
        livresParCode.put(livre.getCode(), livre);
        codesParAuteur.computeIfAbsent(livre.getAuteur(), a -> new ArrayList<>())
            .add(livre.getCode());
    }

    public String titreDe(String code) {
        Livre l = livresParCode.get(code);
        return (l == null) ? null : l.getTitre();
    }
}
```

```

public List<String> titresDe(String auteur) {
    List<String> codes = codesParAuteur.getOrDefault(auteur, List.of());
    List<String> titres = new ArrayList<>();
    for (String c : codes) titres.add(livresParCode.get(c).getTitre());
    return titres;
}

public boolean supprimer(String code) {
    Livre l = livresParCode.remove(code);
    if (l == null) return false;
    List<String> codes = codesParAuteur.get(l.getAuteur());
    codes.remove(code);
    if (codes.isEmpty()) codesParAuteur.remove(l.getAuteur());
    return true;
}
}

```

```

livres de RAKOTO = [Programmation Java, Algorithmique avantee, Bases de donnees]
livres de INCONNU = []      apres suppression = [Programmation Java, Bases de donnees]

```

Trois détails font la différence entre un code juste et un code propre : `computeIfAbsent` remplace « tester, créer, ajouter » en une seule expression; `getOrDefault(auteur, List.of())` évite de renvoyer `null`; `supprimer` montre pourquoi l'invariant compte — retirer le livre du seul index principal laisserait un code fantôme dans l'index inversé, et `titresDe` lèverait une `NullPointerException`.

Exercice 4 — Les essentiels de l'API Stream

Catalogue utilisé : Nokia 2023 (5000 mAh, 6,5", 249 €) · Xiaomi 2024 (5000, 6,7", 399 €) · Samsung 2025 (4500, 6,1", 899 €) · Nokia 2022 (4000, 6,5", 179 €) · Apple 2025 (3300, 6,1", 1099 €).

4.1) Intermédiaire vs terminale

Une opération **intermédiaire** renvoie un nouveau `Stream` et ne calcule rien — `filter`, `map`, `sorted`, `distinct`, `limit`, `skip`. Une opération **terminale** renvoie autre chose qu'un `Stream` — `collect`, `count`, `forEach`, `reduce`, `min`, `max`, `findFirst` — et c'est elle qui déclenche le parcours et consomme le flux. Mnémotechnique : si la méthode rend un `Stream`, elle est intermédiaire.

4.2) Évaluation paresseuse

« Avant `collect` » sort en premier parce que `filter` se contente d'attacher le prédicat au flux : la `lambda` n'est exécutée sur aucun élément. C'est `collect`, l'opération terminale, qui parcourt les éléments et applique le prédicat. « Beta » est absent du résultat parce que le prédicat exige une longueur *strictement* supérieure à quatre — alors même que la ligne `test Beta` s'affiche : l'élément a été testé, ce qui ne veut pas dire qu'il a été retenu.

Conséquence : un pipeline sans opération terminale ne fait strictement rien. Avantage : les éléments traversent le pipeline *un par un*, ce qui permet à `limit(3)` d'interrompre le parcours dès le troisième élément trouvé.

4.3) Un flux ne se parcourt qu'une fois

Non. Un `Stream` n'est pas une collection : c'est un *parcours*. La première opération terminale le consomme et le ferme définitivement; toute opération ultérieure lève `IllegalStateException`: `stream has already been operated upon or closed`. L'échec est immédiat et bruyant — une exception, pas un résultat faux. Lorsqu'on a besoin du résultat plusieurs fois, on le matérialise une seule fois dans une collection :

```

List<Smartphone> abordables = catalogue.stream().filter(p -> p.getPrix() < 500).toList();
long nombre = abordables.size();           // autant de lectures que l'on veut

```

Une collection se relit; un flux se consomme.

4.4) Pourquoi un `Optional`

Parce que le flux peut être vide : après un `filter` qui n'a rien retenu, il n'existe aucun « premier élément » à renvoyer. Renvoyer `null` obligerait chaque appelant à y penser; `Optional` rend l'absence *visible dans la signature*.

```

Smartphone p = catalogue.stream().filter(s -> s.getPrix() > 2000)
    .findFirst().orElse(null);           // valeur de repli
catalogue.stream().findFirst().ifPresentOrElse(System.out::println,

```

```
() -> System.out.println("catalogue vide")); // on agit si presente
// dangereux : .get() sans verification -> NoSuchElementException si le flux est vide
```

4.5) Lien avec l'interface fonctionnelle

L'argument de `filter` est un `Predicate<Smartphone>`, interface fonctionnelle dont l'unique méthode abstraite est `boolean test(T t)`; la lambda `p -> p.getPrix() < 500` en est une implémentation écrite comme une valeur. `filter` est donc une fonction d'ordre supérieur (1.h) qui attend une interface fonctionnelle générique (1.a et 1.d), implémentée par une lambda (1.f).

4.6 à 4.16) Les pipelines

```
// 4.6 -- moins de 600 EUR, du plus recent au plus ancien -> List<Smartphone>
catalogue.stream().filter(p -> p.getPrix() < 600)
                .sorted(Comparator.comparingInt(Smartphone::getAnnee).reversed()).toList();
// [Xiaomi (2024) - 399.00 EUR, Nokia (2023) - 249.00 EUR, Nokia (2022) - 179.00 EUR]
// 4.7 -- marques distinctes par ordre alphabetique -> List<String>
catalogue.stream().map(Smartphone::getMarque).distinct().sorted().toList(); // [Apple, Nokia, Samsung, Xiaomi]
// 4.8 -- prix moyen, puis le plus cher -> OptionalDouble, Optional<Smartphone>
catalogue.stream().mapToDouble(Smartphone::getPrix).average(); // OptionalDouble[565.0]
catalogue.stream().max(Comparator.comparingDouble(Smartphone::getPrix)); // Optional[Apple (2025)...]
// 4.9 -- nombre de modeles de 2025 ou apres -> long
catalogue.stream().filter(p -> p.getAnnee() >= 2025).count(); // 2
// 4.10 -- marques distinctes en une chaine -> String
catalogue.stream().map(Smartphone::getMarque).distinct().sorted()
                .collect(Collectors.joining(", ")); // Apple, Nokia, Samsung, Xiaomi
// 4.11 -- capacite batterie cumulee -> int
catalogue.stream().map(Smartphone::getBatterie).reduce(0, Integer::sum); // 21800
catalogue.stream().mapToInt(Smartphone::getBatterie).sum(); // 21800 (a preferer)
// 4.12 -- les trois quantificateurs -> boolean
catalogue.stream().anyMatch(p -> p.getPrix() > 1000); // true
catalogue.stream().allMatch(p -> p.getBatterie() >= 3000); // true
catalogue.stream().noneMatch(p -> p.getAnnee() < 2020); // true
// 4.13 -- les trois regroupements : groupe seul, comptage, moyenne
catalogue.stream().collect(Collectors.groupingBy(Smartphone::getMarque));
catalogue.stream().collect(Collectors.groupingBy(Smartphone::getMarque, Collectors.counting()));
catalogue.stream().collect(Collectors.groupingBy(Smartphone::getMarque,
                Collectors.averagingDouble(Smartphone::getPrix)));
// {Apple=1099.0, Xiaomi=399.0, Samsung=899.0, Nokia=214.0} <- ordre non garanti
// 4.14 -- de la boucle au pipeline -> List<String>
catalogue.stream().filter(p -> p.getAnnee() >= 2024)
                .map(p -> p.getMarque().toUpperCase()).sorted().toList(); // [APPLE, SAMSUNG, XIAOMI]
// 4.15 -- auteur -> titres, en une instruction (exercice 3)
livres.stream().collect(Collectors.groupingBy(Livre::getAuteur,
                Collectors.mapping(Livre::getTitre, Collectors.toList())));
// {RASOA=[Analyse numerique], RAKOTO=[Programmation Java, Algorithmique avancee, ...]}
```

4.11 — `mapToInt` évite d'emballer chaque `int` dans un `Integer`; `reduce` garde son intérêt quand l'opération de combinaison n'est pas fournie, et sans valeur initiale il renvoie un `Optional`. 4.12 — sur un flux vide, `allMatch` vaut `true` et `anyMatch` `false`. 4.13 — `groupingBy` construit un `HashMap` : l'ordre des clés n'est pas garanti; la surcharge à trois arguments accepte `TreeMap::new`. Son second argument est le *collecteur aval* : il décrit ce qu'on fait de chaque groupe.

4.14 — la correspondance est terme à terme : le `if` devient `filter`, la transformation `map`, le `Collections.sort` final `sorted`, l'accumulateur disparaît. Une différence de contrat : la liste rendue par `toList()` est **non modifiable** (add lève `UnsupportedOperationException`).

4.16) Laquelle choisir si la bibliothèque évolue

La version de l'exercice 3 : le pipeline reconstruit *intégralement* la Map à chaque appel, en $O(n)$, quand la double indexation coûte $O(1)$ par modification. Le pipeline est excellent pour *produire une vue* ponctuelle — rapport, affichage, export — et mauvais comme stockage permanent. La Map de l'exercice 3 est un état que l'on maintient; celle de la question 4.15 est une photographie que l'on recalcule.

Les six réflexes à retenir

- Un flux ne se parcourt qu'une fois.
- Rien ne s'exécute sans opération terminale.
- Un `Optional` se traite, sans `.get()`.
- Un pipeline produit une valeur, il ne remplit rien.
- Pour les nombres : `mapToInt` / `mapToDouble`.
- `groupingBy` rend un `HashMap` non ordonné.